SeraBot — BIMU4101 Midterm Project Report

Nov 6, 2026 · @anıl

SeraBot is an original differential-drive robot that patrols a simulated greenhouse, avoids obstacles and stops to inspect ripe tomatoes. It integrates three sensor types (2D LiDAR, RGB camera and IMU), which meets the requirement for a two-person group.

1. Group and repository

Member	Student number	GitHub
Ahmet Anıl Çay	1306220017	anilcay
Enes Çelik	1306220032	enescelik00

Repository: github.com/anilcay/Mobile-Robotics-Midterm-Project

Environment: Ubuntu 26.04, ROS 2 Lyrical and Gazebo Jetty (gz-sim 10.5.0), the versions named in the assignment. The project was run on native Ubuntu 26.04 under WSL2 and in a Docker image based on `ros:lyrical`.

2. Task and scenario

SeraBot inspects tomato plants in a 10 × 6 m greenhouse. It drives along the aisles between three plant beds, avoids walls, beds and pots, finds red tomatoes and stops beside them for an inspection.

The scenario gives each sensor a real job:

- **LiDAR** keeps the robot clear of obstacles and centred in an aisle.
- **Camera** finds ripe, red produce at plant height.
- **IMU** measures how far the robot has turned, so turns do not depend on wheel odometry.

3. Robot design

The robot is our own design, modelled from scratch in URDF/Xacro with meshes made in Blender. No ready-made robot model is used.

Appearance. The body is a terracotta flowerpot with a soil top. The camera mast is a green plant stem, the camera housing is a flower bud and the LiDAR sits in a pebble-shaped dome. The full visual model has about 15,600 triangles.

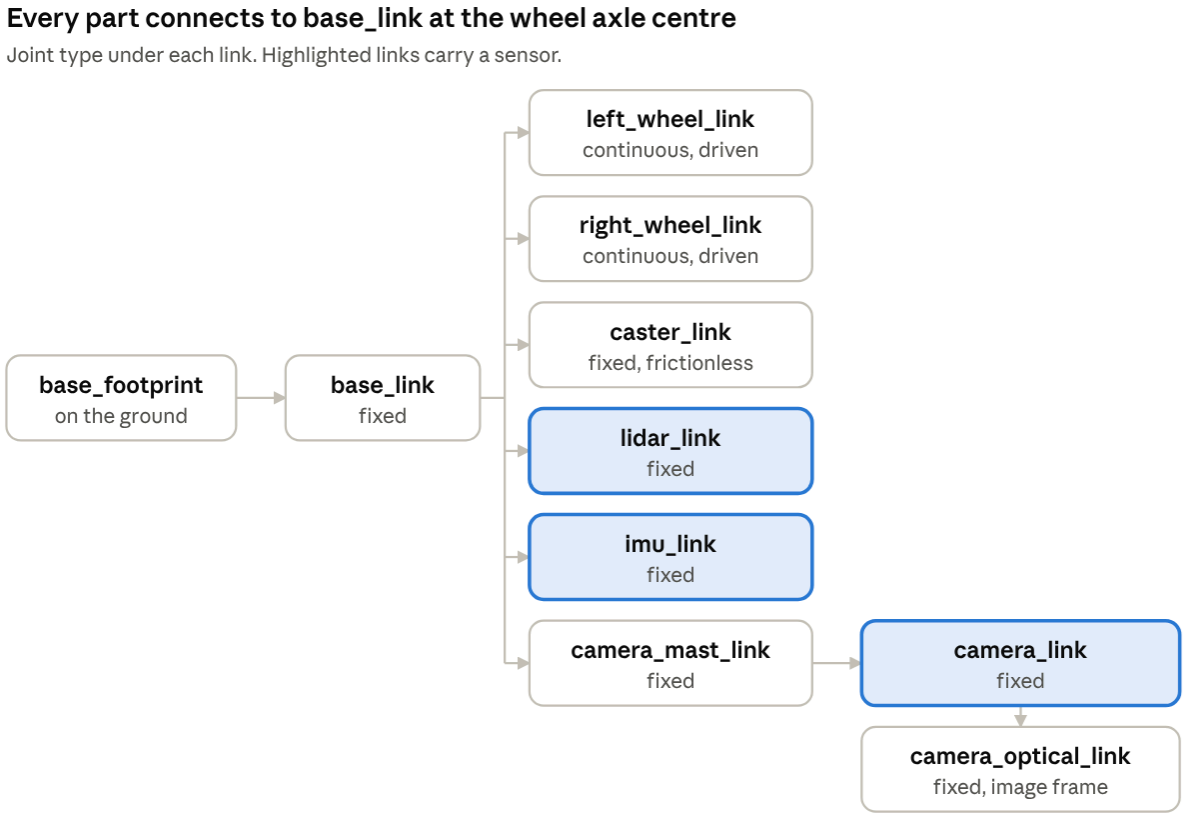


Locomotion. SeraBot uses differential drive: two driven rear wheels and one frictionless front caster. We chose it because the robot can turn in place in aisles only 1.0–1.25 m wide. The Gazebo `DiffDrive` system moves the robot when commands arrive on `/cmd_vel` (`geometry_msgs/msg/Twist`), through `ros_gz_bridge`.

Dimension	Value
Chassis (L × W × H)	0.45 × 0.30 × 0.12 m
Chassis mass	6.0 kg
Wheel radius, width	0.08 m, 0.04 m
Wheel separation	0.36 m
Caster	sphere, radius 0.03 m
Camera height above ground	0.44 m
LiDAR scan plane above ground	0.18 m
Collision width (wheel spheres)	0.52 m

`base_link` sits at the centre of the wheel axle. This is the robot's rotation centre, so odometry and in-place turns stay physically consistent. Collision geometry uses simple primitives while the visuals use the Blender meshes, which keeps the physics stable.

Component connections. The two wheel joints are the only moving joints; everything else is fixed to the body.



Component connections: links and joints of the robot

4. Sensors

The robot has three sensor types, the minimum for a two-person group. Joint states and drive odometry are not counted as sensors.

Sensor	Gazebo type	ROS 2 topic	Message type	Rate (sim time)	Visualization
2D LiDAR	`gpu_lidar`	`/scan`	`sensor_msgs/msg/LaserScan`	10 Hz, 360 samples, 0.25–8 m	RViz LaserScan
RGB camera	`camera`	`/camera/image_raw`, `/camera/camera_info`	`sensor_msgs/msg/Image`, `sensor_msgs/msg/CameraInfo`	15 Hz, 640 × 480, 60° horizontal FOV	RViz image panels, `rqt_image_view`
IMU	`imu`	`/imu`	`sensor_msgs/msg/Imu`	100 Hz	`rqt_plot` via `/imu/rpy`

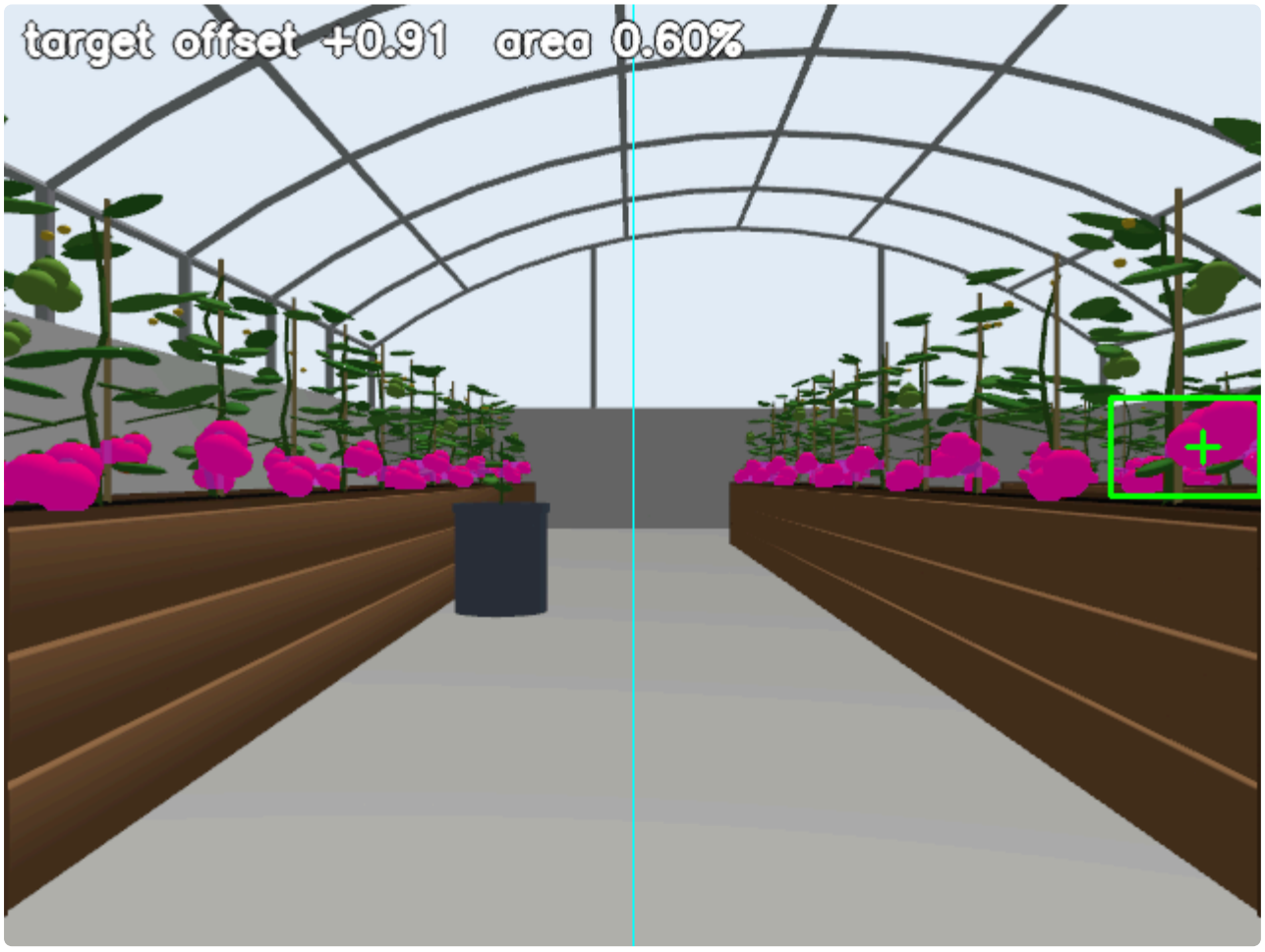
Placement

- **LiDAR, on top of the front of the chassis, 0.18 m above ground.** The scan plane clears the chassis and wheels, and it hits the walls (1.0 m), beds (0.40 m) and pots (0.35 m). The 0.25 m minimum range keeps the mast and the LiDAR housing out of the scan.
- **Camera, on the mast, 0.44 m above ground, level.** Tomatoes hang at about 0.45–0.53 m, so they appear in the middle band of the image. The mast is near the rotation centre, so an in-place turn mostly rotates the view.
- **IMU, at the rotation centre.** During an in-place turn it measures no centripetal acceleration, and its axes match the robot's axes.

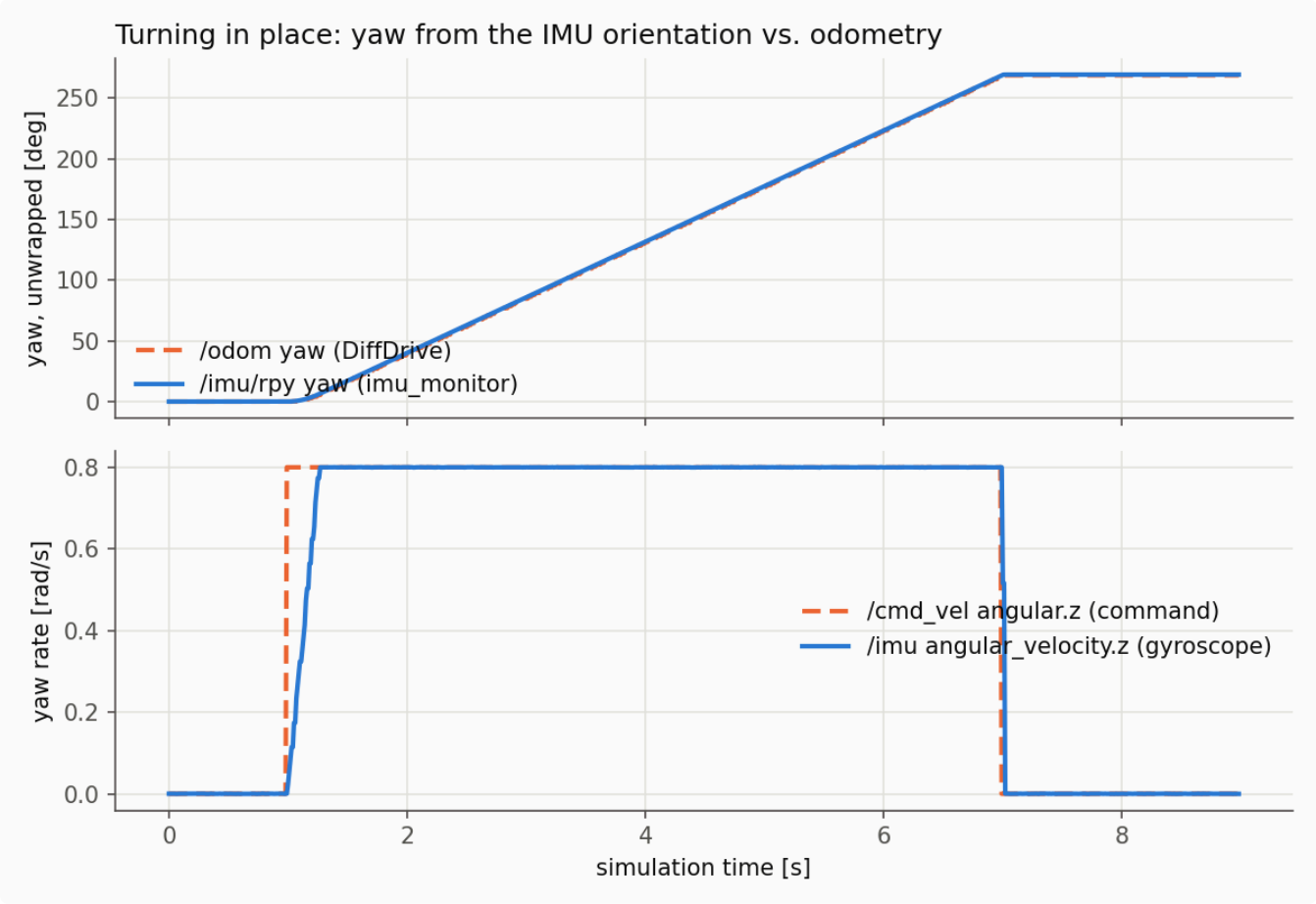
Evidence of data reading

Sensor	Check	Measured
LiDAR	Rate and frame	10.00 Hz (sim time), `frame_id` `lidar_link`
LiDAR	Ranges against the world layout	Mean error −1.0 mm over 342 rays
LiDAR	Self-detection	No reading below 0.30 m
Camera	Rate and frame	15.15 Hz (sim time), `frame_id` `camera_optical_link`
Camera	Calibration	fx = fy = 554.4, cx = 320, cy = 240
Camera	Red detection at spawn	Target offset +0.908, area ratio 0.0060, no false detection on beds or pots

Sensor	Check	Measured
IMU	Rate and frame	100.00 Hz (sim time), `frame_id` `imu_link`
IMU	At rest	`linear_acceleration.z` mean 9.800 m/s²
IMU	Turn commanded at 0.8 rad/s	`angular_velocity.z` mean 0.8000 rad/s
IMU	Yaw against odometry	Largest difference 1.56°



The detector marks red pixels and boxes the largest cluster as the target.



During a commanded in-place turn, IMU yaw follows odometry and the gyro rate follows the command.

Simulation ran slower than real time, so `ros2 topic hz` shows lower wall-clock rates. The table gives rates from message timestamps.

To add before submission: `ros2 topic hz` and `ros2 topic echo` terminal output, and an RViz LaserScan screenshot.

5. Contribution table

Each student owns one sensor, and the additional sensor (IMU) is shared with a clear split.

Sensor	Integration	Data reading	Visualization	Responsible
2D LiDAR	Ahmet Anıl Çay	Ahmet Anıl Çay	Ahmet Anıl Çay	Ahmet Anıl Çay
RGB camera	Enes Çelik	Enes Çelik	Enes Çelik	Enes Çelik

Sensor	Integration	Data reading	Visualization	Responsible
IMU (additional, shared)	Ahmet Anıl Çay	Ahmet Anıl Çay (validation), Enes Çelik (`imu_monitor`)	Enes Çelik	Shared

Other work:

- **Ahmet Anıl Çay:** robot model, greenhouse world and launch file, behavior node.
- **Enes Çelik:** red detector, camera layer of the behavior, demo tuning, README installation and usage.

6. ROS 2 packages and nodes

The workspace has five packages, and one launch file starts the whole system:

```
ros2 launch serabot_bringup bringup.launch.py behavior:=true use_camera:=true
```

Package	Contents
`serabot_description`	Robot model: URDF/Xacro, Blender meshes, model viewer launch file
`serabot_gazebo`	Greenhouse world: primitive collision geometry and Blender decorations
`serabot_bringup`	Single launch file, bridge configuration, node parameters, RViz configuration
`serabot_sensors`	`color_detector` and `imu_monitor` nodes
`serabot_behavior`	`behavior_node` and its ROS-independent decision logic

Node	Subscribes	Publishes	Role
`color_detector`	`/camera/image_raw`	`/camera/target`, `/camera/image_annotated`	Finds the largest red blob with HSV thresholds

Node	Subscribes	Publishes	Role
`imu_monitor`	`/imu`	`/imu/rpy`	Converts orientation to roll, pitch and yaw in degrees for plotting
`behavior_node`	`/scan`, `/imu`, `/camera/target`	`/cmd_vel`, `/behavior/state`	Decides the robot's motion

`/camera/target` is a `geometry_msgs/msg/PointStamped`: `point.x` is the horizontal offset from −1 (left) to +1 (right) and `point.y` is the image area ratio. `ros_gz_bridge` connects Gazebo and ROS 2 topics, and `robot_state_publisher` publishes the robot's transforms.

7. Sensor-based behavior node

`behavior_node` is a Python state machine that runs at 10 Hz in simulation time. It turns all three sensors into one motion command on `/cmd_vel`.

Direction	Topic	Message type	Use
Input	`/scan`	`sensor_msgs/msg/LaserScan`	Distances ahead and to each side, room to turn
Input	`/imu`	`sensor_msgs/msg/Imu`	Turned angle, from integrating `angular_velocity.z`
Input (camera layer)	`/camera/target`	`geometry_msgs/msg/PointStamped`	Red target offset and area
Output	`/cmd_vel`	`geometry_msgs/msg/Twist`	`linear.x` and `angular.z` for the drive
Output	`/behavior/state`	`std_msgs/msg/String`	Current state, for the demo and logs

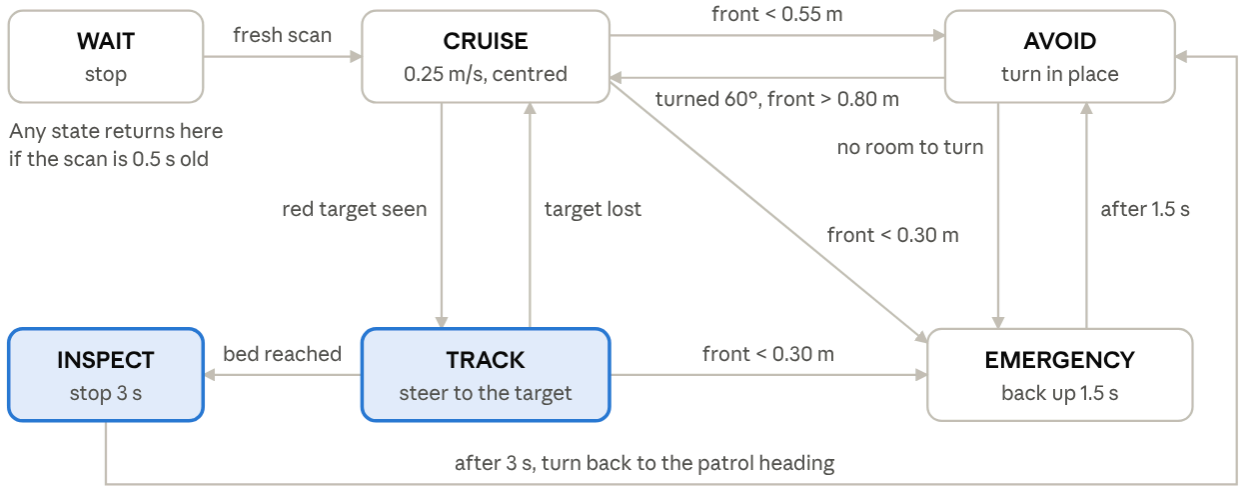
Decision rule

`front_min` is the nearest LiDAR reading in the ±30° front sector or in the corridor the body will drive through.

State	Command	Moves on when
WAIT	Stop	A fresh scan arrives. Any state returns here if the scan is older than 0.5 s
CRUISE	0.25 m/s forward, steering to stay centred in the aisle	`front_min` < 0.30 m → EMERGENCY; `front_min` < 0.55 m → AVOID; red target seen → TRACK
AVOID	Turn in place at 0.8 rad/s towards the more open side with room to turn	Turned at least 60° and `front_min` > 0.80 m → CRUISE; no room on either side → EMERGENCY
EMERGENCY	Back up at 0.1 m/s if the rear is clear	After 1.5 s → AVOID
TRACK	0.12 m/s forward, `angular.z` = −0.8 · offset	Target lost → CRUISE; bed reached or target large enough → INSPECT; `front_min` < 0.30 m → EMERGENCY
INSPECT	Stop for 3 s	Then AVOID, turning back to the patrol heading; targets are ignored for 20 s

LiDAR distance moves the robot between states; the camera adds two

Highlighted states run only with use_camera:=true. Distances are the front_min reading.



State machine of behavior_node

Before a turn, the node checks the circle swept by the body (radius 0.376 m around `base_link`), so the front corners do not hit an obstacle. If the IMU stops publishing, the turned angle falls back to a time-based estimate.

The camera layer (TRACK and INSPECT) is optional. With `use_camera:=false` the robot still avoids obstacles using only the LiDAR and IMU. The decision logic is separate from ROS and is covered by unit tests.

8. Simulation results

The robot completed every recorded run without a collision and without entering EMERGENCY in the final configuration. Clearance is the smallest distance between the robot body and any wall, bed or pot, computed from Gazebo's ground-truth pose.

Robot motion

- **Driving straight:** at 0.25 m/s the robot moved 0.618 m in Gazebo with no sideways drift, and `/odom` reported 0.613 m.
- **Turning in place:** at 0.8 rad/s the gyro integral was 268.2° against 268.9° expected.

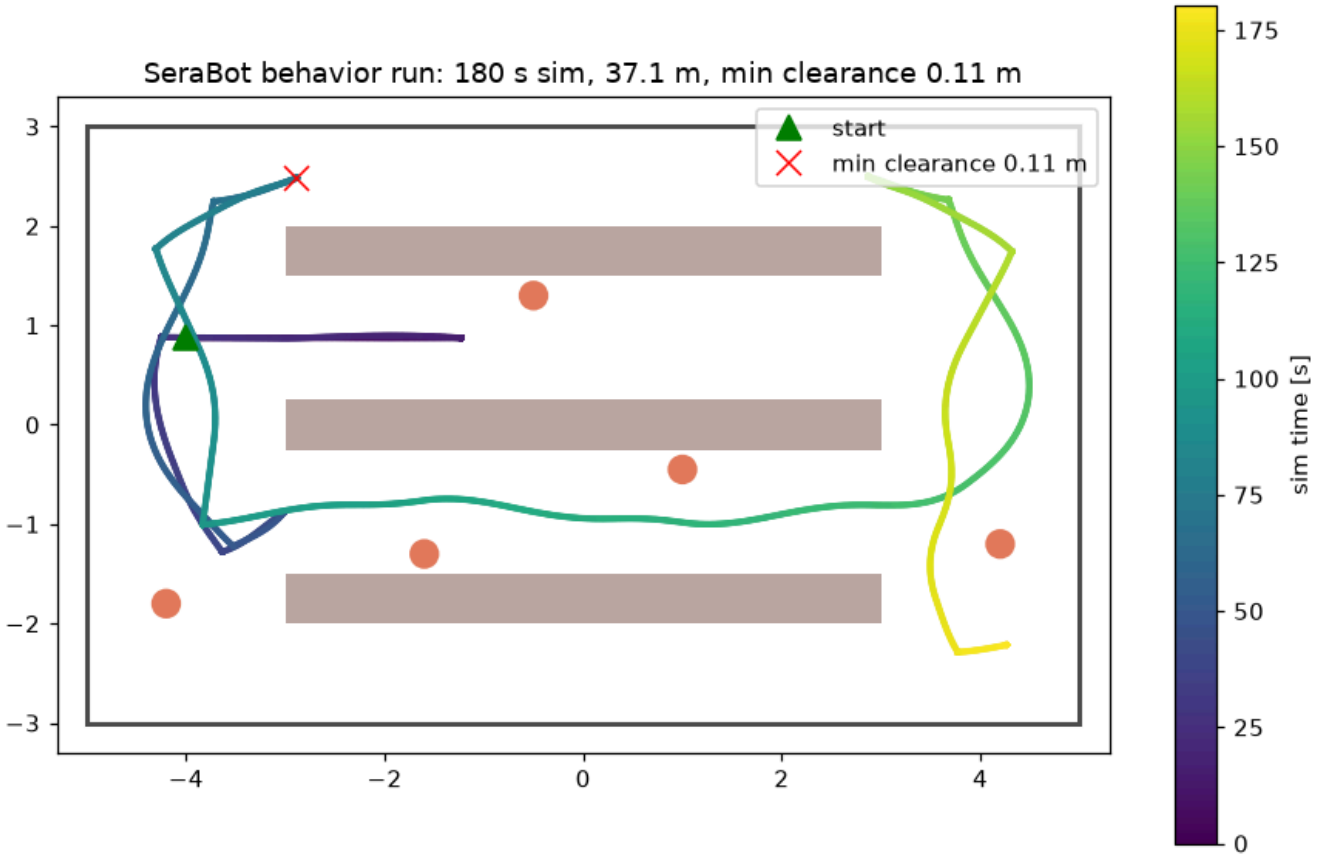
Effect of sensor data on behavior

The node logs the reason for each state change, so the link from sensor to motion is visible:

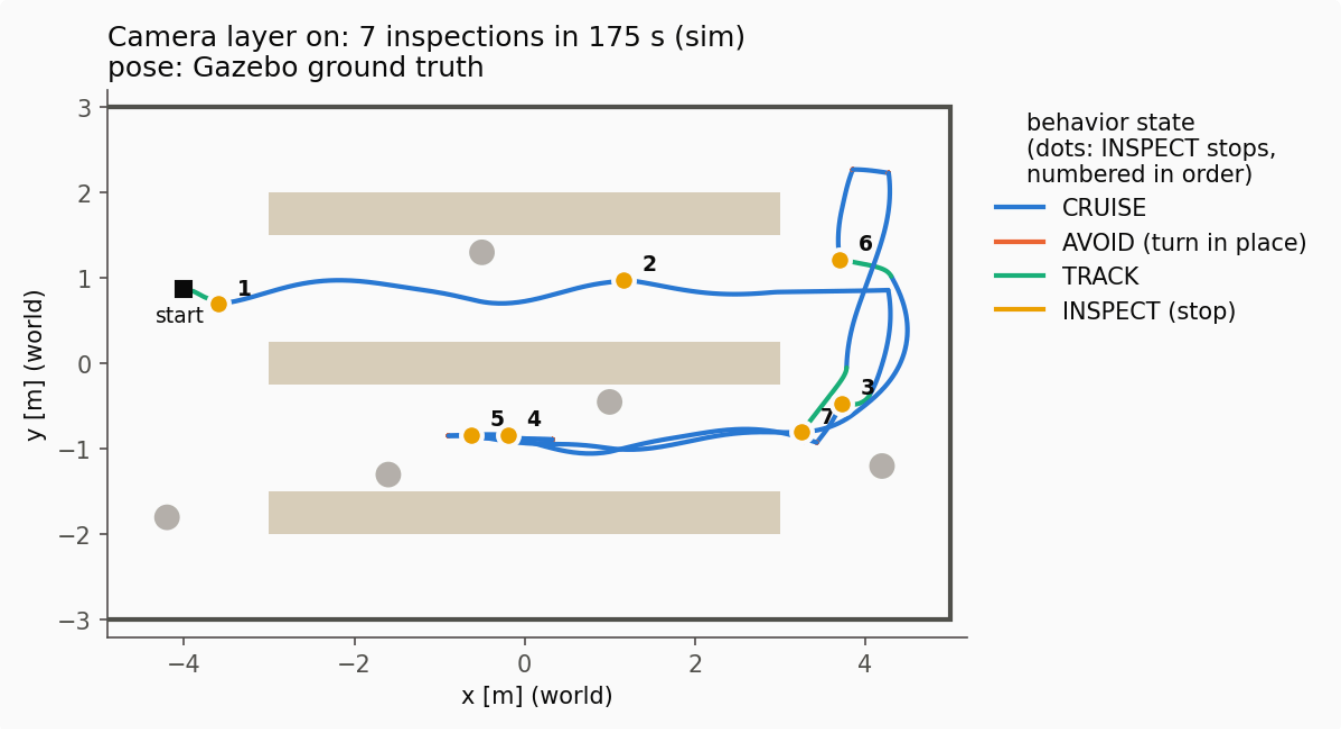
```
CRUISE -> AVOID: front 0.55 m < 0.55 m; turning right
AVOID -> CRUISE: turned 176 deg (imu), front 0.98 m
```

With the camera layer on, the first target appears at offset +0.91. The robot steers right until the offset falls to +0.05, stops at the bed for 3 s, then turns back to its patrol.

Run	Sim time	Visited cells (1 × 1 m)	Inspections	Smallest clearance	EMERGENCY
Obstacle avoidance, run 3	180 s	29/60	not used	0.106 m	0
Obstacle avoidance, repeat 1	180 s	31/60	not used	0.103 m	0
Obstacle avoidance, repeat 2	180 s	29/60	not used	0.119 m	0
Camera layer	180 s	21/60	7	0.128 m	0
Camera layer, full demo command	180 s	20/60	7	0.123 m	0
Camera layer, long run	300 s	23/60	12	0.114 m	0



Obstacle avoidance run 3: the ground-truth path through the greenhouse in 180 s of simulation.



Camera layer: the same aisles, now with seven numbered inspection stops.

The camera changes the path. The first camera version kept the robot in the west turning area (9/60 cells), because each inspection pointed it at the next truss. Returning to the patrol heading after an inspection raised coverage to 21/60 cells, with both aisles travelled end to end.

A fresh clone of the repository built with `colcon build`, passed 61 tests with no failures and ran a full TRACK and INSPECT cycle.

9. Known limitations and conclusion

SeraBot meets the project goals: it moves through ROS 2 commands, reads and shows three sensor types, and changes its motion based on sensor data. Four limitations remain:

- **Odometry drift:** `/odom` drifted by 3.34 m in the 300 s run. The behavior does not use `/odom`, so its decisions are not affected.
- **Long camera runs:** the robot can move back and forth between the two pots of the south-middle aisle. It did not collide or lock up.
- **Inspection trigger:** the target area reached at most 1.5% of the image in the first camera tests, so reaching the bed, not the 8% area threshold, starts INSPECT.
- **Gazebo shutdown:** Gazebo 10.5.0 can crash with a segmentation fault when it closes. This was not seen while the simulation was running.